

LAPORAN PRATIKUM PEMROGRAMAN

PRATIKUM 5

STATEMENT PERULANGAN



Disusun Oleh :

NAMA	: Muhammad Aditya Rahmansyah
NRP	: 3226600122
KELAS	: ST.r Tekkom A
MATA KULIAH	: Pratikum Pemrograman
DOSEN	: Ir. Sigit Wasista, M.Kom.
ASSISTEN DOSEN	: Rofiulloh Mu'azin Al-insof, A.Md.
TANGGAL PRATIKUM	: Rabu, 30 September 2026

PROGRAM STUDI SARJANA TERAPAN TEKNIK KOMPUTER
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

PENDAHULUAN

5.1 Tujuan Praktikum

Tuliskan tujuan praktikum berdasarkan modul, misalnya:

1. Menjelaskan konsep perulangan dan alasan penggunaannya dalam program.
2. Membedakan karakteristik `for`, `while`, dan `do-while`.
3. Menggunakan `for` untuk perulangan dengan pola penghitung yang jelas.
4. Menggunakan `while` untuk kondisi yang diperiksa sebelum perulangan.
5. Menggunakan `do-while` untuk proses yang minimal dijalankan satu kali.
6. Menerapkan `nested loop`, `break`, dan `continue`.
7. Menguji program dengan beberapa data masukan dan menjelaskan hasilnya.

5.2 Dasar Teori

Jelaskan secara singkat:

- Pengertian looping.

Jawaban :

sekumpulan instruksi secara berulang selama kondisi tertentu terpenuhi.

- Fungsi looping dalam pemrograman.

Jawaban :

Looping menghindari penulisan kode yang sama berkali-kali

- `for`

Jawaban :

Menjalankan blok kode secara berulang kali secara efisien ketika jumlah perulangan sudah diketahui

- `while`

Jawaban :

perulangan yang digunakan untuk menjalankan sekumpulan blok kode secara berulang-ulang selama syarat atau kondisi tertentu bernilai benar

- `do-while`

Jawaban :

struktur kode untuk menjalankan perintah secara berulang, di mana pengecekan kondisi dilakukan di akhir blok perulangan

- `nested loop`

Jawaban :

perulangan di mana terdapat perulangan di dalam perulangan lain.

- `break`

Jawaban :

dalam C++ adalah perintah khusus untuk menghentikan dan keluar dari perulangan (for, while, do-while) atau percabangan (switch) secara paksa sebelum waktunya

- `continue`

Jawaban :

digunakan untuk melewati sisa kode pada iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya di dalam perulangan

Catatan: Jangan menyalin seluruh teori dari modul. Cukup 1–2 halaman dan gunakan bahasa sendiri.

5.5 PERCOBAAN TERBIMBING

Buat format yang sama untuk setiap percobaan...

Percobaan 1 – Perulangan `for`

A. Tujuan

1. Memahami cara kerja perulangan `for` dalam bahasa C.
2. Memahami tiga bagian `for`: inisialisasi, kondisi, dan perubahan nilai (increment).
3. Menampilkan bilangan berurutan dari 1 sampai 5 menggunakan perulangan.

B. Source Code

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    return 0; }
```

C. Algoritma

Tuliskan langkah algoritma dalam bentuk nomor:

1. Mulai.
2. Inisialisasi variabel $i = 1$.
3. Periksa kondisi $i \leq 5$.
4. Jika benar, tampilkan nilai i .
5. Tambahkan i sebesar 1.
6. Kembali ke pemeriksaan kondisi.
7. Jika kondisi salah, program selesai.

D. Hasil Percobaan

Screenshot source code:

```
#include <stdio.h>

int main(void) {
    int i = 1;

    while (i <= 5) {
        printf("%d ", i);
        i++;
    }

    return 0;
}
```

Screenshot hasil program:

```
1 2 3 4 5
```

E. Analisis

Jelaskan:

- Apa fungsi $i = 1$? Memberi nilai awal sebelum perulangan dimulai.
- Apa fungsi $i \leq 5$? Untuk cek apakah nilai i kurang atau sama dengan 5.
- Apa fungsi $i++$? Menambah nilai i sebanyak 1.
- Berapa kali perulangan dilakukan? 5 kali.

- Apa yang terjadi jika `i++` dihilangkan? Nilai `i` akan tetap 1, tidak bertambah atau berkurang sehingga kondisi `i <= 5` akan selalu benar sehingga akan terjadi infinite loop.

Percobaan – Tringular

A. Tujuan

1. Memahami cara kerja perulangan for dalam bahasa C.
2. Memahami penggunaan variabel akumulator untuk menyimpan hasil penjumlahan berulang.
3. Menghitung jumlah bilangan berurutan dari 1 sampai 200 menggunakan perulangan.

B. Algoritma

Tuliskan langkah algoritma dalam bentuk nomor:

1. Mulai.
2. Inisialisasi variabel jumlah = 0.
3. Inisialisasi variabel n = 1.
4. Periksa kondisi `n <= 200`.
5. Jika benar, tambahkan nilai n ke jumlah (`jumlah += n`).
6. Tambahkan n sebesar 1.
7. Kembali ke pemeriksaan kondisi.
8. Jika kondisi salah, tampilkan nilai jumlah.
9. Program selesai.

C. Source Code

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int n, jumlah = 0;
```

```
    for (n = 1; n <= 200; n++) {
```

```

        jumlah += n;

    }

    printf("Jumlah 200 triangular = %d\n", jumlah);

    return 0;

}

```

D. Hasil percobaan

```

#include <stdio.h>

int main(void) {
    int n, jumlah = 0;

    for (n = 1; n <= 200; n++) {
        jumlah += n;
    }

    printf("Jumlah 200 triangular = %d\n", jumlah);

    return 0;
}

```

E. Screenshoot hasil :

```
Jumlah 200 triangular = 20100
```

E. Analisis

Pada program ini, jumlah = 0 memberi nilai awal pada variabel penampung hasil sebelum penjumlahan dimulai agar tidak berisi nilai sampah, sedangkan n = 1 memberi nilai awal pada counter sebelum perulangan dimulai. Kondisi n <= 200 dicek sebelum setiap putaran untuk memastikan nilai n masih kurang dari atau sama dengan 200, dan n++ menambah nilai n sebanyak 1 di setiap putaran sehingga counter bergerak menuju batas. Di dalam perulangan, jumlah += n menambahkan nilai n saat ini ke jumlah (sama dengan jumlah = jumlah + n), sehingga setelah perulangan dilakukan sebanyak 200 kali, jumlah berisi hasil penjumlahan 1 sampai 200, yaitu 20100. Jika n++ dihilangkan, nilai n akan tetap 1 dan tidak bertambah, sehingga kondisi n <= 200 selalu benar dan terjadi infinite loop.

Percobaan – Nested Loop

A. Permasalahan

Membuat program yang mencetak pola segitiga siku-siku dari tanda bintang (*) sebanyak 5 baris. Baris pertama berisi 1 bintang, baris kedua 2 bintang, dan seterusnya sampai baris kelima berisi 5 bintang. Masalah ini membutuhkan **perulangan di dalam perulangan** (nested loop) karena jumlah bintang per baris berubah mengikuti nomor barisnya.

B. Algoritma

1. Mulai.
2. Ulangi untuk *i* dari 1 sampai 5 (mengatur baris):
 1. Ulangi untuk *j* dari 1 sampai *i* (mengatur kolom):
 - Tampilkan * .
 2. Setelah loop dalam selesai, pindah baris ($\backslash n$).
3. Selesai.

C. Source Code

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int i, j;
```

```
    for (i = 1; i <= 5; i++) {
```

```
        for (j = 1; j <= i; j++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

D. Pengujian

I = 5

E. Hasil Percobaan

```
Segitiga kiri:
*
* *
* * *
* * * *
* * * * *
```

F. Analisa

Program memakai nested loop: loop luar (i) mengatur baris dan berjalan 5 kali, sedangkan loop dalam (j) mengatur jumlah bintang pada satu baris dan selalu dimulai dari 1 lagi di setiap baris baru. Kunci polanya ada pada batas loop dalam $j \leq i$: karena i bertambah tiap baris, jumlah bintang ikut bertambah (1, 2, 3, 4, 5) sehingga terbentuk segitiga siku-siku. Jika diganti $j \leq 5$, setiap baris berisi 5 bintang dan polanya menjadi persegi. `printf("\n")` berada di luar loop dalam tetapi di dalam loop luar, sehingga pindah baris dilakukan sekali setelah satu baris selesai; jika dipindah ke dalam loop dalam, setiap bintang akan berada di baris sendiri, dan jika dihapus semua bintang tercetak dalam satu baris. Total bintang yang dicetak adalah $1 + 2 + 3 + 4 + 5 = 15$ dan mengikuti rumus $n(n+1)/2$.

Percobaan – break

A. Tujuan

1. Memahami fungsi `break` untuk menghentikan seluruh perulangan ketika kondisi tertentu terpenuhi.
2. Mengetahui penggunaan `break` pada perulangan `for`.
3. Menghentikan perulangan ketika nilai `i` mencapai 5.

B. Algoritma

1. Mulai.
2. Inisialisasi variabel `i = 1`.

3. Periksa kondisi $i \leq 10$.
4. Jika benar, tampilkan nilai i .
5. Periksa apakah $i == 5$.
6. Jika $i == 5$, tampilkan pesan bahwa perulangan dihentikan, kemudian jalankan `break`.
7. Jika kondisi tidak terpenuhi, tambahkan i sebesar 1.
8. Kembali ke pemeriksaan kondisi.
9. Jika `break` dijalankan atau kondisi $i \leq 10$ salah, program selesai.

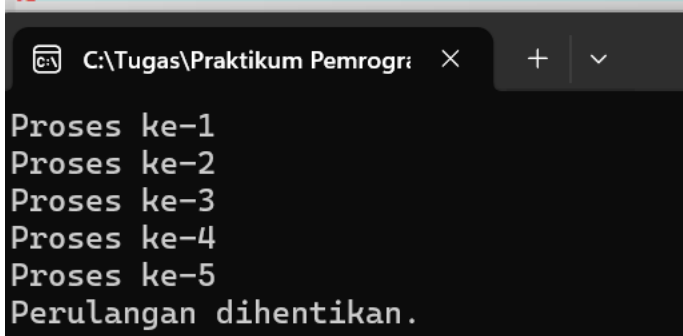
```
#include <stdio.h>

int main(void) {
    int i;

    for (i = 1; i <= 10; i++) {
        printf("Proses ke-%d\n", i);

        if (i == 5) {
            printf("Perulangan dihentikan.\n");
            break;
        }
    }

    return 0;
}
```



```
C:\Tugas\Praktikum Pemrogr: X + v
Proses ke-1
Proses ke-2
Proses ke-3
Proses ke-4
Proses ke-5
Perulangan dihentikan.
```

C. Analisis

Pada program ini, perulangan `for` dimulai dari $i = 1$ sampai $i = 10$. Setiap putaran menampilkan nilai i , kemudian kondisi `if (i == 5)` memeriksa apakah i bernilai 5. Saat i sama dengan 5, perintah `break` dijalankan sehingga seluruh perulangan langsung dihentikan. Oleh karena itu, output hanya sampai `Proses ke-5` dan perulangan tidak dilanjutkan ke angka berikutnya. Berbeda dengan `continue` yang hanya melewati satu putaran, `break` menghentikan seluruh perulangan.

Percobaan - continue

A. Tujuan

1. Memahami fungsi `continue` untuk melewati sisa isi perulangan pada putaran saat ini, lalu lanjut ke putaran berikutnya.
2. Membedakan `continue` (hanya melewati satu putaran) dengan `break` (menghentikan seluruh perulangan).
3. Menampilkan bilangan 1 sampai 10 dengan melewati angka 5.

B. Algoritma

1. Mulai.
2. Inisialisasi variabel `i = 1`.
3. Periksa kondisi `i <= 10`.
4. Jika benar, periksa apakah `i == 5`.
5. Jika `i == 5`, jalankan `continue` (lewat `printf`, langsung ke langkah 7).
6. Jika bukan, tampilkan nilai `i`.
7. Tambahkan `i` sebesar 1.
8. Kembali ke pemeriksaan kondisi.
9. Jika kondisi salah, program selesai.

C. Source Code

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int i;
```

```
    for (i = 1; i <= 10; i++) {
```

```
        if (i == 5) {
```

```
            continue;
```

```
        }
```

```
        printf("%d ", i);
```

```

    }

    return 0;
}

```

D. Hasil Percobaan

```
1 2 3 4 6 7 8 9 10
```

E. Analisis

Pada program ini, perulangan for berjalan dari $i = 1$ sampai $i = 10$. Di setiap putaran, kondisi `if (i == 5)` memeriksa apakah i bernilai 5. Saat i sama dengan 5, perintah `continue` dijalankan sehingga `printf` pada putaran itu dilewati dan program langsung lanjut ke $i++$, akibatnya angka 5 tidak ditampilkan. Perulangan tidak berhenti dan tetap berjalan sampai $i = 10$, sehingga output menjadi 1 2 3 4 6 7 8 9 10. Ini berbeda dengan `break` yang akan menghentikan seluruh perulangan, sehingga jika `continue` diganti `break`, output hanya 1 2 3 4. Perulangan tetap berjalan 10 kali, tetapi hanya 9 angka yang dicetak.

5.6 LATIHAN TERBIMBING

Buat format yang sama untuk setiap latihan.

1. Latihan `for` – Menjumlahkan Bilangan 1 sampai n

A. Permasalahan

Membuat program yang menerima bilangan bulat n dari user, lalu menghitung total dari $1 + 2 + 3 + \dots + n$. Contoh: $n = 5 \rightarrow 1 + 2 + 3 + 4 + 5 = 15$.

B. Algoritma

1. Input nilai n .

2. Inisialisasi `jumlah = 0`.
3. Lakukan perulangan dari 1 sampai `n`.
4. Tambahkan setiap nilai ke `jumlah`.
5. Tampilkan hasil.

C. Source Code

```
#include <stdio.h>

int main(void) {

    int n, i, total;

    printf("Masukkan nilai n: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n harus bilangan bulat positif.\n");
    } else {
        total = 0;

        for (i = 1; i <= n; i++) {
            total = total + i;
        }

        printf("Jumlah 1 sampai %d adalah %d\n", n, total);
    }

    return 0;
```

}

D. Pengujian

No	Input	output	keterangan
1	5	Jumlah 1 sampai 5 adalah 15	Sesuai
2	10	Jumlah 1 sampai 10 adalah 55	Sesuai
3	-1	n harus bilangan bulat positif.	Sesuai

E. Screenshot

```
#include <stdio.h>

int main(void) {
    int n, i, total;

    printf("Masukkan nilai n: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n harus bilangan bulat positif.\n");
    } else {
        total = 0;

        for (i = 1; i <= n; i++) {
            total = total + i;
        }

        printf("Jumlah 1 sampai %d adalah %d\n", n, total);
    }

    return 0;
}
```

```
Masukkan nilai n: 5
Jumlah 1 sampai 5 adalah 15
```

```
Masukkan nilai n: 10
Jumlah 1 sampai 10 adalah 55
```

```
Masukkan nilai n: -1
n harus bilangan bulat positif.
```

F. Analisis

Program ini meminta user memasukkan bilangan bulat n , lalu mengecek apakah n positif; jika nol atau negatif, program langsung menampilkan pesan error dan berhenti. Jika valid, variabel total diset ke 0 sebagai penampung, kemudian perulangan for berjalan dengan i dari 1 sampai n , dan di setiap putaran nilai i ditambahkan ke total ($\text{total} = \text{total} + i$). Setelah i melewati n , loop berhenti dan program mencetak isi total sebagai hasil penjumlahan 1 sampai n . Pada pengujian, input 5 menghasilkan 15, input 10 menghasilkan 55, input 1 menghasilkan 1, serta input -1 menghasilkan pesan error, dan semua hasil itu sesuai dengan rumus $n(n+1)/2$ sehingga program terbukti benar untuk input valid maupun tidak valid.

2. Latihan while – Validasi Nilai

Gunakan struktur:

A. Permasalahan

Membuat program yang meminta user memasukkan nilai dalam rentang **0 sampai 100**. Jika input di luar rentang, program terus meminta input ulang sampai nilainya valid. Pada modifikasi ini, program juga **menghitung berapa kali user melakukan input** sebelum nilai diterima.

B. Algoritma

1. Mulai.
2. Set `percobaan = 1`.
3. Minta user memasukkan nilai, simpan ke `nilai`.
4. Periksa kondisi `nilai < 0 || nilai > 100`.
5. Jika benar (tidak valid): tampilkan pesan error, tambah `percobaan` sebesar 1, minta input lagi, lalu kembali ke langkah 4.
6. Jika salah (valid): keluar dari loop.
7. Tampilkan nilai yang diterima dan jumlah percobaan.
8. Selesai.

C. Source Code

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int nilai;
```

```

int percobaan = 1;

printf("Masukkan nilai 0-100: ");

scanf("%d", &nilai);

while (nilai < 0 || nilai > 100) {

    printf("Nilai tidak valid. Masukkan lagi: ");

    scanf("%d", &nilai);

    percobaan++;

}

printf("Nilai diterima = %d\n", nilai);

printf("Jumlah percobaan input = %d\n", percobaan);

return 0;

}

```

D. Pengujian

No	Input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	85	Nilai diterima = 85; percobaan = 1	Nilai diterima = 85; percobaan = 1	Valid di percobaan pertama, loop tidak jalan
2	150 90	Nilai diterima = 90; percobaan = 2	Nilai diterima = 90; percobaan = 2	Satu kali salah, pada nilai 150

3	-5 101 70	Nilai diterima = 70; percobaan = 3	Nilai diterima = 70; percobaan = 3	Dua kali salah, pada nilai -5 dan 101
4	0	Nilai diterima = 0; percobaan = 1	Nilai diterima = 0; percobaan = 1	Batas bawah valid
5	100	Nilai diterima = 100; percobaan = 1	Nilai diterima = 100; percobaan = 1	Batas atas valid

E. Screenshoot

```
#include <stdio.h>

int main(void) {

    int nilai;
    int percobaan = 1;

    printf("Masukkan nilai 0-100: ");
    scanf("%d", &nilai);

    while (nilai < 0 || nilai > 100) {
        printf("Nilai tidak valid. Masukkan lagi: ");
        scanf("%d", &nilai);
        percobaan++;
    }

    printf("Nilai diterima = %d\n", nilai);
    printf("Jumlah percobaan input = %d\n", percobaan);

    return 0;
}
```

```
Masukkan nilai 0-100: 85
Nilai diterima = 85
Jumlah percobaan input = 1
```

```
Masukkan nilai 0-100: 150 90
Nilai tidak valid. Masukkan lagi: Nilai diterima = 90
Jumlah percobaan input = 2
```

```
Masukkan nilai 0-100: -5 101 70
Nilai tidak valid. Masukkan lagi: Nilai tidak valid. Masukkan lagi: Nilai diterima = 70
Jumlah percobaan input = 3
```

```
Masukkan nilai 0-100: 0          Masukkan nilai 0-100: 100
Nilai diterima = 0              Nilai diterima = 100
Jumlah percobaan input = 1      Jumlah percobaan input = 1
```

F. Analisis

Perulangan while cocok untuk validasi karena jumlah pengulangan tidak diketahui di awal, bergantung pada seberapa sering user salah input. Kondisi dicek sebelum badan loop dijalankan, sehingga jika input pertama sudah valid, loop dilewati sepenuhnya.

Kondisi nilai < 0 || nilai > 100 memakai operator || karena nilai tidak valid jika berada di bawah 0 atau di atas 100. Memakai && salah, karena tidak ada angka yang sekaligus kurang dari 0 dan lebih dari 100, sehingga loop tidak pernah jalan.

Penghitung percobaan diawali 1 karena scanf pertama (sebelum loop) sudah satu percobaan. Setiap pengulangan loop berarti satu percobaan tambahan. Jadi jumlah percobaan = jumlah salah input + 1.

Latihan ini menggunakan validasi nilai 0–100 sebagaimana pada modul.

3. Latihan do-while – Menu Sederhana

Gunakan struktur:

A. Permasalahan

Membuat program menu yang tampil berulang sampai user memilih keluar. Menu menyediakan tiga pilihan: menampilkan pesan, menampilkan angka 1-5, atau keluar (0). Menu harus tampil minimal satu kali, dan input yang tidak tersedia harus ditolak dengan pesan, bukan menghentikan program.

B. Algoritma

1. Mulai.
2. Tampilkan menu.
3. Baca pilihan.
4. Jika `pilihan == 1`, tampilkan "Praktikum Pemrograman Dasar".
5. Jika tidak, jika `pilihan == 2`, tampilkan "1 2 3 4 5".
6. Jika tidak, jika `pilihan != 0`, tampilkan "Pilihan tidak tersedia."
7. Periksa kondisi `pilihan != 0`.
8. Jika benar, kembali ke langkah 2.
9. Jika salah, keluar dari loop.
10. Selesai.

C. Source Code

```
#include <stdio.h>

int main(void) {

    int pilihan;

    do {

        printf("\n=== MENU ===\n");

        printf("1. Tampilkan pesan\n");

        printf("2. Tampilkan angka 1-5\n");

        printf("0. Keluar\n");

        printf("Pilihan: ");

        scanf("%d", &pilihan);

        if (pilihan == 1)

            printf("Praktikum Pemrograman Dasar\n");

        else if (pilihan == 2)

            printf("1 2 3 4 5\n");

        else if (pilihan != 0)

            printf("Pilihan tidak tersedia.\n");

    } while (pilihan != 0);

    return 0;
```

}

D. Pengujian

No	Input	Hasil yang diharapkan	Hasil aktual	keterangan
1	1	Praktikum Pemrograman Dasar	Praktikum Pemrograman Dasar	Pilihan 1
2	2	1 2 3 4 5	1 2 3 4 5	Pilihan 2
3	0	Langsung keluar tanpa pesan apa pun	Langsung keluar tanpa pesan apa pun	Pilihan keluar
4	5	Pilihan tidak tersedia.	Pilihan tidak tersedia.	Input di luar menu

E. Screenshoot

```
#include <stdio.h>
int main(void) {

    int pilihan;

    do {
        printf("\n=== MENU ===\n");
        printf("1. Tampilkan pesan\n");
        printf("2. Tampilkan angka 1-5\n");
        printf("0. Keluar\n");
        printf("Pilihan: ");
        scanf("%d", &pilihan);

        if (pilihan == 1)
            printf("Praktikum Pemrograman Dasar\n");
        else if (pilihan == 2)
            printf("1 2 3 4 5\n");
        else if (pilihan != 0)
            printf("Pilihan tidak tersedia.\n");
    } while (pilihan != 0);

    return 0;
}
```

```
=== MENU ===
1. Tampilkan pesan
2. Tampilkan angka 1-5
0. Keluar
Pilihan: 1
Praktikum Pemrograman Dasar

=== MENU ===
1. Tampilkan pesan
2. Tampilkan angka 1-5
0. Keluar
Pilihan: 2
1 2 3 4 5

=== MENU ===
1. Tampilkan pesan
2. Tampilkan angka 1-5
0. Keluar
Pilihan: 5
Pilihan tidak tersedia.

=== MENU ===
1. Tampilkan pesan
2. Tampilkan angka 1-5
0. Keluar
Pilihan: 0
```

F. Analisis

Perulangan do-while cocok untuk menu karena badan loop dijalankan dulu, baru kondisi dicek. Dengan begitu menu pasti tampil minimal satu kali, berbeda dengan while yang bisa saja tidak berjalan sama sekali.

Tidak ada perintah khusus untuk keluar. Loop berhenti ketika kondisi pilihan $\neq 0$ bernilai salah, yaitu saat user memasukkan 0. Pada saat itu, ketiga cabang if / else if dilewati, sehingga tidak ada pesan yang tercetak.

Cabang else if (pilihan $\neq 0$) berfungsi menangkap input yang tidak tersedia. Pengecekan $\neq 0$ diperlukan agar angka 0, yang merupakan perintah keluar, tidak ikut dianggap pilihan salah.

Perhatikan bahwa menu harus terus ditampilkan sampai pengguna memilih 0.

4. Latihan Nested Loop – Pola Segitiga

Gunakan struktur:

A. Permasalahan

Membuat program yang mencetak pola segitiga siku-siku dari tanda bintang (*) sebanyak 5 baris. Baris pertama berisi 1 bintang, baris kedua 2 bintang, dan seterusnya sampai baris kelima berisi 5 bintang. Masalah ini membutuhkan **perulangan di dalam perulangan** (nested loop) karena jumlah bintang per baris berubah mengikuti nomor barisnya.

B. Algoritma

1. Mulai.
2. Ulangi untuk *i* dari 1 sampai 5 (mengatur baris):
 1. Ulangi untuk *j* dari 1 sampai *i* (mengatur kolom):
 - Tampilkan * .
 2. Setelah loop dalam selesai, pindah baris ($\backslash n$).
3. Selesai.

C. Source Code

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int i, j;
```

```
    for (i = 1; i <= 5; i++) {
```

```
        for (j = 1; j <= i; j++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```

return 0;

}

```

D. Pengujian

No	Jumlah baris	Hasil yang diharapkan	Hasil aktual	keterangan
1	5	<pre> * * * * * * * * * * * * * * * </pre>	<pre> * * * * * * * * * * * * * * * </pre>	Sesuai Modul
2	3	<pre> * * * * * * </pre>	<pre> * * * * * * </pre>	Pola menyesuaikan batas
3	1	<pre> * </pre>	<pre> * </pre>	Kasus terkecil

E. Screenshoot

```

#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 1; i <= 5; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}

```

```

*
* *
* * *
* * * *
* * * * *

```

```
#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 1; i <= 3; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}
```

```
*
* *
* * *
```

```
#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 1; i <= 1; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    return 0;
}
```

```
*
```

F. Analisis

Peran masing-masing loop

- Loop luar (i) mengatur baris. Pada kode asli ia berjalan 5 kali, satu putaran per baris.
- Loop dalam (j) mengatur jumlah bintang pada satu baris. Ia selalu dimulai dari 1 lagi tiap baris baru, dan berhenti di $j \leq i$.
- `printf("\n")` berada di luar loop dalam tetapi di dalam loop luar, sehingga pindah baris dilakukan sekali setelah satu baris bintang selesai.

Kunci polanya adalah batas loop dalam $j \leq i$. Karena i bertambah tiap baris, jumlah bintang ikut bertambah. Kalau diganti $j \leq 5$, setiap baris akan berisi 5 bintang (persegi, bukan segitiga).

5.7 TUGAS PRAKTIKUM

Modul memberikan **10 tugas praktikum**, dan setiap program harus disertai **source code**, **contoh input-output**, serta **penjelasan algoritma**.

Tugas 1 – Matriks Identitas

Soal:

Buat program untuk mencetak matriks identitas berukuran $n \times n$.

Algoritma

1. Mulai.
2. Baca nilai n (ukuran matriks).
3. Ulangi untuk i dari 1 sampai n (baris):
 1. Ulangi untuk j dari 1 sampai n (kolom):
 - Jika $i == j$, tampilkan 1.
 - Jika tidak, tampilkan 0.
 2. Setelah satu baris selesai, pindah baris.
4. Selesai.

Source Code:

```
#include <stdio.h>

int main(void) {
    int n, i, j;

    printf("Masukkan ukuran matriks (n): ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n harus bilangan bulat positif.\n");
    } else {
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                if (i == j)
                    printf("1 ");
                else
                    printf("0 ");
            }
            printf("\n");
        }
    }

    return 0;
}
```

Input:

5

Output:

```
Masukkan ukuran matriks (n): 5
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
```

Analisis:

Loop luar (i) mengatur baris dan loop dalam (j) mengatur kolom, sehingga setiap pasangan (baris, kolom) dikunjungi tepat satu kali, total $n \times n$ kali. Di dalam loop dalam, kondisi $i == j$ menentukan isi elemen: jika nomor baris sama dengan nomor kolom, berarti elemen tersebut berada di diagonal utama sehingga dicetak 1, selain itu dicetak 0. `printf("\n")` diletakkan di luar loop dalam tetapi di dalam loop luar agar pindah baris dilakukan sekali setiap satu baris selesai dicetak. Validasi $n \leq 0$ mencegah program berjalan dengan ukuran matriks yang tidak masuk akal. Program ini tidak memakai array karena matriks hanya perlu dicetak, bukan disimpan. Kalau nanti dosen meminta matriks disimpan, bisa memakai array 2 dimensi `int m[n][n]` dengan isi yang ditentukan oleh kondisi $i == j$ yang sama.

Tugas 2 – Tabel Perkalian 1–10

Soal:

Buat program menggunakan `nested loop` untuk menampilkan tabel perkalian 1 sampai 10.

Algoritma

1. Mulai.
2. Ulangi untuk i dari 1 sampai 10 (baris):
 1. Ulangi untuk j dari 1 sampai 10 (kolom):
 - Hitung $i * j$, lalu tampilkan.
 2. Setelah satu baris selesai, pindah baris.
3. Selesai.

Source Code:

```
#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 1; i <= 10; i++) {
        for (j = 1; j <= 10; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }

    return 0;
}
```

Input:

10

Output:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Analisis:

Loop luar (i) mewakili bilangan pengali di setiap baris, dan loop dalam (j) mewakili kolom 1 sampai 10, sehingga setiap kombinasi i dan j dihitung tepat satu kali, total $10 \times 10 = 100$ perkalian. Pada setiap putaran loop dalam, $i * j$ dihitung dan dicetak dengan %4d agar setiap angka memakai lebar 4 karakter dan kolom tampak rata, termasuk angka 100 yang terdiri dari tiga digit. printf("\n") diletakkan di luar loop dalam tetapi di dalam loop luar agar pindah baris dilakukan sekali setelah satu baris selesai. Tabelnya simetris terhadap diagonal utama karena $i * j$ sama dengan $j * i$. Nilai terbesar adalah $10 \times 10 = 100$, jauh di bawah batas int, sehingga tidak ada risiko overflow.

Tugas 3 – Membalikkan Digit

Contoh:

Input : 1234

Output : 4321

Algoritma:

1. Mulai.
2. Baca bilangan angka.
3. Set balik = 0.
4. Selama angka > 0:
 1. Ambil digit terakhir: $\text{digit} = \text{angka} \% 10$.
 2. Susun ke hasil: $\text{balik} = \text{balik} * 10 + \text{digit}$.
 3. Buang digit terakhir: $\text{angka} = \text{angka} / 10$.
5. Tampilkan balik.
6. Selesai.

Source Code:

```
#include <stdio.h>

int main(void) {

    int angka, asli, digit, balik = 0;

    printf("Masukkan bilangan bulat: ");
    scanf("%d", &angka);

    if (angka < 0) {
        printf("Masukkan bilangan tidak negatif.\n");
    } else {
        asli = angka;

        while (angka > 0) {
            digit = angka % 10;
            balik = balik * 10 + digit;
            angka = angka / 10;
        }

        printf("Input : %d\n", asli);
        printf("Output : %d\n", balik);
    }

    return 0;
}
```

Pengujian:

Input Output

```
Masukkan bilangan bulat: 1234
Input : 1234
Output : 4321
```

Analisis:

Program ini memakai perulangan while karena jumlah digit tidak diketahui di awal. Operator % dengan 10 mengambil digit terakhir, sedangkan pembagian integer / 10 membuang digit terakhir tersebut, sehingga setiap putaran memproses satu digit dari belakang ke depan.

Variabel balik disusun dengan rumus balik $\times 10 + \text{digit}$: mengalikan 10 menggeser digit yang sudah ada satu tempat ke kiri, lalu digit baru masuk di posisi satuan, sehingga digit yang diambil pertama (paling belakang) berakhir di posisi paling depan. Nilai awal balik = 0 wajib ada agar hasil tidak berisi nilai sampah. Bilangan yang berakhir dengan nol, seperti 1200, menghasilkan 21 karena bilangan bulat tidak menyimpan nol di depan. Variabel asli dipakai untuk menyimpan input awal karena angka berubah selama loop. Keterbatasannya, bilangan yang besar (misalnya 1999999999) bisa overflow saat dibalik karena hasilnya melebihi batas int, dan solusinya memakai long long.

Tugas 4 – Konversi Suhu

Buat tabel konversi:

- Celsius → Fahrenheit
- Celsius → Reamur
- Celsius → Kelvin

Source Code:

```
#include <stdio.h>

int main(void) {

    float c, f, r, k;

    printf("Masukkan suhu Celsius: ");
    scanf("%f", &c);

    f = (c * 9.0 / 5.0) + 32;
    r = c * 4.0 / 5.0;
    k = c + 273.15;

    printf("=====\n");
    printf("| Celsius    -> Fahrenheit = %8.2f |\n", f);
    printf("| Celsius    -> Reamur    = %8.2f |\n", r);
    printf("| Celsius    -> Kelvin    = %8.2f |\n", k);
    printf("=====\n");

    return 0;
}
```

Pengujian:

```
Masukkan suhu Celsius: 32
=====
| Celsius    -> Fahrenheit =    89.60 |
| Celsius    -> Reamur     =    25.60 |
| Celsius    -> Kelvin     =   305.15 |
=====
```

Analisis:

Program ini menerima satu nilai suhu Celsius dari user, lalu menghitung tiga konversi sekaligus memakai rumus Fahrenheit = $(C \times 9/5) + 32$, Reamur = $C \times 4/5$, dan Kelvin = $C + 273.15$. Hasil sesuai titik acuan: air membeku di $0^{\circ}\text{C} = 32^{\circ}\text{F} = 0^{\circ}\text{R} = 273.15\text{ K}$ dan mendidih di $100^{\circ}\text{C} = 212^{\circ}\text{F} = 80^{\circ}\text{R} = 373.15\text{ K}$, sedangkan pada -40°C nilai Celsius dan Fahrenheit sama. Program ini tidak memakai perulangan karena hanya mengonversi satu nilai per eksekusi.

Tugas 5 – Jumlah dan Rata-rata n Nilai

Source Code:

```
#include <stdio.h>

int main (void) {

    int n, i, nilai, jumlah;
    float rata2;

    printf("Masukkan jumlah data = ");
    scanf("%d", &n);

    if (n <= 0){
        printf("Tidak boleh");
    } else {

        jumlah = 0;

        for (i = 1; i <= n; i++) {
            printf("Masukkan nilai pertama ke-%d = ", i);
            scanf("%d", &nilai);
            jumlah = jumlah + nilai;
        }

        rata2 = jumlah / n;

        printf("Jumlah nilai = %d\n", jumlah);
        printf("Rata - rata nilai = %.2f", rata2);

    }

    return 0;
}
```

Pengujian:

No	Jumlah data	Data	Jumlah	Rata - Rata	Keterangan
1	5	90, 89, 87, 87, 91	444	88.00	Data banyak
2	3	70, 80, 90	240	80	Sesuai soal
3	3	70, 80, 85	235	78.33	Pembagian desimal
4	1	96	96	96	1 data
5	0	-	-	-	Jumlah data tidak boleh 0 dan validasi pesan error

Analisis:

Program menjumlahkan n nilai dengan perulangan `for` dan variabel `jumlah` sebagai penampung. Rata-rata dihitung setelah loop dengan `(float) jumlah / n` agar hasilnya desimal, dan validasi `n <= 0` mencegah pembagian dengan nol.

Tugas 6 – Jumlah Digit Bilangan

Contoh:

Input : 3255
Output : 15

```

#include <stdio.h>

int main(void) {

    int angka, digit, asli, jumlah;

    printf("Masukkan angka = ");
    scanf("%d", &angka);

    if (angka < 0) {
        printf("Angka tidak boleh negatif\n");
    } else {
        asli = angka;
        jumlah = 0;

        while (angka > 0) {
            digit = angka % 10;
            jumlah = jumlah + digit;
            angka = angka / 10;
        }

        printf("Input dari user = %d\n", asli);
        printf("Jumlah = %d\n", jumlah);
    }

    return 0;
}

```

No	Input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	123	Input dari user = 123 Jumlah = 6	Input dari user = 123 Jumlah = 6	Input 3 angka
2	1	Masukkan angka = 1 Input dari user = 1 Jumlah = 1	Masukkan angka = 1 Input dari user = 1 Jumlah = 1	Input 1 angka
3	-1 0	Masukkan angka = -1 0 Angka tidak boleh negatif	Angka tidak boleh negatif	Angka negative dan 0

Analisa :

Program menjumlahkan semua digit sebuah bilangan dengan perulangan while, karena jumlah digit tidak diketahui di awal. Di setiap putaran, angka % 10 mengambil digit terakhir dan ditambahkan ke jumlah, lalu angka / 10 membuang digit tersebut. Perulangan berhenti saat angka menjadi 0. Variabel asli menyimpan input awal agar bisa ditampilkan, karena angka berubah selama loop. Validasi angka < 0 menolak bilangan negatif.

Tugas 7 – Tabel Faktorial

Tampilkan **10 faktorial pertama** dengan format yang rapi.

n n! Hasil

```
1 1! 1
2 2! 2
3 3! 6
... .. 10
10! ...
```

```
#include <stdio.h>

int main(void) {

    int n;
    int faktorial = 1;

    printf("%-5s %-8s %s\n", "n", "n!", "Hasil");

    for (n = 1; n <= 10; n++) {
        faktorial = faktorial * n;
        printf("%-5d ", n);
        printf("%d!", n);
        if (n < 10)
            printf(" ");
        else
            printf(" ");
        printf("%d\n", faktorial);
    }

    return 0;
}
```

n	n!	Hasil
1	1!	1
2	2!	2
3	3!	6
4	4!	24
5	5!	120
6	6!	720
7	7!	5040
8	8!	40320
9	9!	362880
10	10!	3628800

Analisa :

Program menampilkan faktorial 1 sampai 10 dengan satu perulangan for, karena jumlah barisnya sudah diketahui. Variabel faktorial diawali 1 (bukan 0, agar hasil perkalian tidak menjadi 0) dan di setiap putaran dikalikan dengan n (faktorial = faktorial * n), sehingga n! dihitung dari (n-1)! tanpa perlu menghitung ulang dari awal. Perkalian dilakukan sebelum printf agar nilai yang dicetak sudah benar. Tanda ! pada kolom n! dicetak lewat printf("%d!", n), dan lebar kolom pada format printf membuat tabel rata.

Tugas 8 – Minimum, Maksimum, dan Rata-rata

Input n data nilai siswa kemudian tampilkan:

- nilai minimum;
- nilai maksimum;
- rata-rata.

```

#include <stdio.h>

int main(void) {

    int n, i, nilai;
    int min, maks, jumlah;
    float rata;

    printf("Masukkan jumlah data = ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Jumlah data harus lebih dari 0\n");
    } else {

        printf("Masukkan nilai ke-1 = ");
        scanf("%d", &nilai);
        min = nilai;
        maks = nilai;
        jumlah = nilai;

        for (i = 2; i <= n; i++) {
            printf("Masukkan nilai ke-%d = ", i);
            scanf("%d", &nilai);

            jumlah = jumlah + nilai;

            if (nilai < min)
                min = nilai;

            if (nilai > maks)
                maks = nilai;
        }

        rata = (float) jumlah / n;

        printf("Nilai minimum = %d\n", min);
        printf("Nilai maksimum = %d\n", maks);
        printf("Rata-rata = %.2f\n", rata);
    }

    return 0;
}

```

Output :

```

Masukkan jumlah data = 4
Masukkan nilai ke-1 = 80
Masukkan nilai ke-2 = 90
Masukkan nilai ke-3 = 87
Masukkan nilai ke-4 = 98
Nilai minimum = 80
Nilai maksimum = 98
Rata-rata = 88.75

```

Analisa :

Program membaca n nilai dengan perulangan for, karena jumlah data sudah diketahui sebelum loop dimulai. Nilai minimum dan maksimum diinisialisasi dari data pertama (bukan 0 atau angka tebakan) agar benar untuk semua rentang nilai, lalu setiap data berikutnya dibandingkan dan memperbarui min atau maks bila lebih kecil atau lebih besar. Total nilai dihitung dengan akumulator jumlah, dan rata-rata dihitung setelah loop dengan (float) jumlah / n agar pecahan tidak terbuang. Validasi $n \leq 0$ mencegah pembagian dengan nol.

Tugas 9 – break

Buat program pencarian nilai target menggunakan break.

Pengujian minimal 2 kondisi:

1. Target ditemukan.
2. Target tidak ditemukan.

```
#include <stdio.h>

int main(void) {
    int n, i, target, nilai;
    int ditemukan = 0;
    int posisi = 0;

    printf("Masukkan jumlah data = ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Jumlah data harus lebih dari 0\n");
    } else {

        printf("Masukkan nilai target = ");
        scanf("%d", &target);

        for (i = 1; i <= n; i++) {
            printf("Masukkan nilai ke-%d = ", i);
            scanf("%d", &nilai);

            if (nilai == target) {
                ditemukan = 1;
                posisi = i;
                break;
            }
        }
    }
}
```

```

    }

    if (ditemukan == 1)
        printf("Target %d ditemukan pada data ke-%d\n", target, posisi);
    else
        printf("Target %d tidak ditemukan\n", target);
}

return 0;

```

Masukkan jumlah data = 4	Masukkan jumlah data = 4
Masukkan nilai target = 89	Masukkan nilai target = 34
Masukkan nilai ke-1 = 34	Masukkan nilai ke-1 = 57
Masukkan nilai ke-2 = 67	Masukkan nilai ke-2 = 569
Masukkan nilai ke-3 = 45	Masukkan nilai ke-3 = 67
Masukkan nilai ke-4 = 89	Masukkan nilai ke-4 = 24
Target 89 ditemukan pada data ke-4	Target 34 tidak ditemukan

Analisis :

Program membaca data satu per satu dengan perulangan for dan membandingkan setiap nilai dengan target. Saat nilai sama dengan target, variabel ditemukan diset 1, posisinya disimpan, lalu break menghentikan perulangan seketika sehingga data sisanya tidak perlu dibaca. Jika loop selesai tanpa break, ditemukan tetap 0 sehingga program menampilkan bahwa target tidak ditemukan. Variabel penanda ditemukan diperlukan karena setelah loop program harus tahu apakah loop berhenti karena break atau karena data habis.

Tugas 10 – continue

Buat program menggunakan `continue` untuk melewati data yang memenuhi kondisi tertentu.

Jelaskan **mengapa data tersebut dilewati**.

Tujuan

1. Memahami fungsi `continue` untuk melewati sisa isi perulangan pada putaran saat ini, lalu lanjut ke putaran berikutnya.
2. Membedakan `continue` (hanya melewati satu putaran) dengan `break` (menghentikan seluruh perulangan).
3. Membuat program yang memproses data tetapi mengabaikan data yang tidak memenuhi syarat.

Algoritma

1. Mulai.

2. Inisialisasi `jumlah = 0` dan `dihitung = 0`.
3. Baca jumlah data `n`. Jika `n <= 0`, tampilkan pesan error lalu selesai.
4. Ulangi untuk `i` dari 1 sampai `n`:
 1. Baca `nilai`.
 2. Jika `nilai < 0` atau `nilai > 100`, tampilkan pesan data dilewati, lalu **continue**.
 3. `jumlah = jumlah + nilai`.
 4. `dihitung = dihitung + 1`.
5. Jika `dihitung > 0`, tampilkan `dihitung`, `jumlah`, dan rata-rata (`jumlah / dihitung`).
6. Jika tidak, tampilkan bahwa tidak ada data valid.
7. Selesai.

```
#include <stdio.h>

int main(void) {

    int n, i, nilai;
    int jumlah = 0;
    int dihitung = 0;

    printf("Masukkan jumlah data = ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Jumlah data harus lebih dari 0\n");
    } else {

        for (i = 1; i <= n; i++) {
            printf("Masukkan nilai ke-%d = ", i);
            scanf("%d", &nilai);

            if (nilai < 0 || nilai > 100) {
                printf("Nilai %d dilewati (di luar rentang 0-100)\n", nilai);
                continue;
            }

            jumlah = jumlah + nilai;
            dihitung = dihitung + 1;
        }

        if (dihitung > 0) {
            printf("Data valid = %d\n", dihitung);
            printf("Jumlah = %d\n", jumlah);
            printf("Rata-rata = %.2f\n", (float) jumlah / dihitung);
        } else {
            printf("Tidak ada data valid\n");
        }
    }

    return 0;
}
```

```
Masukkan jumlah data = 5
Masukkan nilai ke-1 = 80
Masukkan nilai ke-2 = -5
Nilai -5 dilewati (di luar rentang 0-100)
Masukkan nilai ke-3 = 90
Masukkan nilai ke-4 = 150
Nilai 150 dilewati (di luar rentang 0-100)
Masukkan nilai ke-5 = 70
Data valid = 3
Jumlah = 240
Rata-rata = 80.00
```

Analisa :

Program membaca n nilai dengan perulangan for, dan setiap nilai diperiksa dengan kondisi $\text{nilai} < 0 \parallel \text{nilai} > 100$. Nilai di luar rentang 0-100 dilewati karena bukan nilai ujian yang sah: jika ikut dijumlahkan, nilai seperti -5 atau 150 akan merusak total dan rata-rata. Ketika kondisi itu benar, continue langsung melompat ke putaran berikutnya tanpa menjalankan $\text{jumlah} = \text{jumlah} + \text{nilai}$ dan $\text{dihitung} = \text{dihitung} + 1$, sedangkan perulangan tetap berjalan sampai data ke-n. Hal ini berbeda dengan break yang menghentikan seluruh perulangan. Variabel dihitung dipakai sebagai pembagi rata-rata (bukan n), karena data yang dilewati tidak boleh ikut dihitung, dan pengecekan $\text{dihitung} > 0$ mencegah pembagian dengan nol jika semua data dilewati.

5.8 TANTANGAN PENGEMBANGAN

Kerjakan minimal sesuai instruksi.

1. Triangular

Hitung jumlah triangular dari 1 sampai n menggunakan for.

```
#include <stdio.h>

int main(void) {
    int n, i, jumlah = 0;

    printf("Masukkan n = ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        jumlah += i;
    }

    printf("Jumlah triangular 1 sampai %d = %d\n", n, jumlah);

    return 0;
}
```

```
C:\Tugas\Praktikum Pemrogr
Masukkan n = 100
Jumlah triangular 1 sampai 100 = 5050
```

2. Menu Berulang

Buat menu menggunakan do-while dan berhenti ketika pengguna memilih 0.

```

1  #include <stdio.h>
2
3  int main(void) {
4
5      int pilihan;
6
7      do {
8          printf("\n=== MENU ===\n");
9          printf("1. Halo\n");
10         printf("2. Angka 1-5\n");
11         printf("0. Keluar\n");
12         printf("Pilihan: ");
13         scanf("%d", &pilihan);
14
15         if (pilihan == 1)
16             printf("Halo!\n");
17         else if (pilihan == 2)
18             printf("1 2 3 4 5\n");
19         else if (pilihan != 0)
20             printf("Pilihan tidak tersedia.\n");
21
22     } while (pilihan != 0);
23
24     return 0;
25 }
```

```

=== MENU ===
1. Halo
2. Angka 1-5
0. Keluar
Pilihan: 1
Halo!

=== MENU ===
1. Halo
2. Angka 1-5
0. Keluar
Pilihan: 2
1 2 3 4 5

=== MENU ===
1. Halo
2. Angka 1-5
0. Keluar
Pilihan: 5
Pilihan tidak tersedia.

=== MENU ===
1. Halo
2. Angka 1-5
0. Keluar
Pilihan: 0
```

```

Compile Log
Debug
Find Results
Close
```

```

Output Size: 100 3305070105 74B
```

3. Input Nilai

Program meminta nilai berulang kali sampai pengguna memasukkan -1.

Nilai -1 **tidak dihitung** dalam rata-rata.

```
#include <stdio.h>

int main(void) {
    int nilai, jumlah = 0, banyak = 0;

    printf("Masukkan nilai (-1 untuk selesai)\n");
    scanf("%d", &nilai);

    while (nilai != -1) {
        jumlah += nilai;
        banyak++;
        scanf("%d", &nilai);
    }

    if (banyak > 0)
        printf("Rata-rata = %.2f\n", (float) jumlah / banyak);
    else
        printf("Tidak ada data\n");

    return 0;
}
```

Masukkan nilai (-1 untuk selesai)
34
34
45
45
-1
Rata-rata = 39.50

Process exited after 7.18 seconds
Press any key to continue . . . |

4. Pola dengan Nested Loop

Buat:

1. segitiga kiri;
2. segitiga terbalik;
3. persegi.

```
#include <stdio.h>

int main(void) {
    int i, j, n = 5;

    printf("Segitiga kiri:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }

    printf("\nSegitiga terbalik:\n");
    for (i = n; i >= 1; i--) {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }

    printf("\nPersegi:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }

    return 0;
}
```

Segitiga kiri:

```
*
* *
* * *
* * * *
* * * * *
```

Segitiga terbalik:

```
* * * * *
* * * *
* * *
* *
*
```

Persegi:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Process exited after

5.9 ANALISIS DAN REFLEKSI

Jawab pertanyaan berikut dengan **bahasa sendiri**.

1 Apa perbedaan utama posisi pemeriksaan kondisi pada for, while, dan do-while?

Jawaban :

Pada for dan while, kondisi dicek terlebih dahulu sebelum loop dijalankan. Bedanya pada for menyatukan inisialisasi, kondisi, dan perubahan nilai pada satu baris, sedangkan pada while hanya berisi kondisi dan inisialisasi serta perubahan nilai ditulis terpisah. Pada do while, kondisi diperiksa setelah loop dijalankan.

Pada for dan while, kondisi dicek terlebih dahulu sebelum loop dijalankan. Bedanya pada for menyatukan inisialisasi, kondisi, dan perubahan nilai pada satu baris, sedangkan pada while hanya berisi kondisi dan inisialisasi serta perubahan nilai ditulis terpisah. Pada do while, kondisi diperiksa setelah loop dijalankan.

2 Mengapa `while` dan `for` dapat tidak menjalankan badan loop sama sekali?

Jawaban : Karena kondisinya dicek terlebih dahulu baru setelah itu badan loop dijalankan. Jika kondisi salah, maka program tidak menjalankan loop dan melewatinya.

3 Berikan contoh kasus nyata yang lebih sesuai menggunakan `do-while` daripada `while`.

Jawaban :

Menu program, seperti menu ATM atau menu pada praktikum ini. Menu harus tampil minimal satu kali agar pengguna bisa memilih, baru setelah itu program mengecek apakah pengguna memilih keluar (0).

4 Apa risiko perulangan tak hingga dan bagaimana cara mencegahnya?

Jawaban :

Perulangan tak hingga membuat program tidak pernah berhenti, sehingga terlihat hang, menghabiskan CPU, dan bisa memenuhi layar dengan output. Cara mencegah :

1. Pastikan ada perubahan nilai misal `i++` sampai menuju kondisi berhenti
2. Hati – hati saat menulis `=` atau `==`
3. Beri validasi input
4. Gunakan `break`

5 Jelaskan perbedaan fungsi `break` dan `continue`.

Jawaban :

`break` menghentikan seluruh perulangan dan program lanjut ke perintah setelah loop. `continue` hanya melewati sisa isi loop pada putaran saat ini, lalu lanjut ke putaran berikutnya.

6 Pada `nested loop`, variabel mana yang mengendalikan baris dan mana yang mengendalikan kolom?

Jawaban :

Loop luar (variabel `i`) mengendalikan baris, dan loop dalam (variabel `j`) mengendalikan kolom.

KESIMPULAN

Tuliskan kesimpulan berdasarkan hasil praktikum.

Berdasarkan hasil praktikum, looping digunakan untuk menjalankan perintah secara berulang sehingga program lebih singkat. `for` digunakan saat jumlah perulangan diketahui, `while` saat kondisi menentukan perulangan, dan `do-while` saat perintah harus dijalankan minimal sekali. Nested loop digunakan untuk membuat pola atau tabel, sedangkan `break` menghentikan loop dan `continue` melewati satu putaran. Hasil pengujian menunjukkan program berjalan sesuai kondisi dan input yang diberikan.

FORMAT UNTUK SETIAP PROGRAM

Supaya laporan **seragam dan mudah diperiksa**, setiap program sebaiknya wajib menggunakan urutan:

1. Soal → 2. Tujuan → 3. Algoritma → 4. Source Code → 5. Input → 6. Output → 7. Screenshot → 8. Analisis

sesuai dengan kriteria modul yaitu **pemahaman konsep, ketepatan algoritma/logika, kebenaran kode, pengujian dan analisis hasil, serta kerapian laporan.**

Checklist sebelum dikumpulkan

- Semua program dapat dikompilasi.
- Tidak ada infinite loop yang tidak disengaja.
- Setiap program diuji dengan lebih dari satu kasus.
- Output sesuai spesifikasi.
- Source code rapi.
- Ada screenshot hasil eksekusi.
- Ada algoritma.
- Ada analisis hasil.
- Mahasiswa memahami kode yang dibuat.